Can AI help keep data validation rules up to date?

Maintaining and extending expert rules when data standards change

MSc Artificial Intelligence thesis proposal

Tomash Oosterbaan
University of Groningen | Avanade and Rabobank
6 November 2026

Overview

Imagine a data team at a bank that checks the reports it must send to its supervisor. The reports follow a standard that changes regularly: new items are added, and old items are renamed or removed. The hundreds of validation rules that check the reports must follow these changes. A rule that is not updated may quietly stop checking the data it was meant to check, or it may reject data that is correct under the new standard. Today, people find and update the affected rules by hand. At the same time, the team keeps discovering kinds of error for which no rule exists yet.

This thesis builds and evaluates a platform that helps with both tasks and makes every change testable before it goes live. When a new version of a standard is published, a change radar reads the publisher's documentation of the change and lists the rules that are probably affected, with the reason. A local language model proposes an updated version of each affected rule, and the platform tests it on example data. For kinds of error that no rule covers, the platform inserts typical errors into a safe copy of the data, shows which ones slip through, and lets the model propose new rules for exactly those gaps. Every new or changed rule then starts in shadow mode: it runs on incoming data, but its alerts are only recorded. The platform replays several years of historical data to show what the rule would have caught and how many alerts it would have raised. A person reviews this evidence and decides whether the rule may block data, may only warn, or should only be monitored. The approved rules run as a quality gate in a data pipeline, and every decision is recorded for auditors.

The research behind the platform asks whether these steps can be trusted: whether a model finds and updates the right rules when a standard changes, whether new rules add anything beyond the existing ones, and whether that still holds on real data over time. To answer these questions against an independent reference, the thesis uses public rule sets with a published version history. An industry committee publishes a version of its rules for each yearly version of the US accounting taxonomy, and the European Banking Authority publishes its validation rules for each release of its reporting framework. The experts' own updates are the correct answers. No bank data is needed.

1 Problem and research objective

Data teams check incoming data before it is used in reports, analyses or other systems. These checks range from simple technical controls, such as required fields and valid codes, to business rules that relate several fields to each other. At Rabobank, this work includes technical validation, business rules, monitoring thresholds and the manual investigation of exceptions [1]. Banks must also be able to show that such controls exist and work: the Basel Committee's principles for risk data (BCBS 239) and the European Central Bank's 2024 guide treat data quality and its controls as part of sound risk management [2, 3].

Most of these controls are validation rules. A validation rule is a condition that every record must meet, for example "total assets must equal total liabilities plus equity" in a balance sheet. When

a record does not meet the condition, the rule raises an alert, and a person investigates whether the data is wrong. Writing good rules requires knowledge of the data and of the business.

Validation rules are also not written once. They refer to the items of a reporting standard, and these standards change. The US GAAP taxonomy, the dictionary of concepts in which US companies report their financial statements, is published every year by the Financial Accounting Standards Board (FASB), together with files that list which concepts are deprecated and which concepts replace them [4]. The XBRL US Data Quality Committee (DQC) therefore keeps a separate version of its rules for each taxonomy year [5], and the European Banking Authority (EBA) publishes a new set of validation rules with each release of its reporting framework, with a label for every rule that states what changed [6]. In the pilot work for this thesis, the DQC changed the logic of 62 rule parts and added 56 new ones in three yearly updates, and the EBA changed the formula of 1,094 rules at least once between releases (Section 2.7). A rule that is not updated can stop checking the data it was meant to check, because it looks for a concept that is no longer reported, or it can raise alerts on data that is correct under the new version. The internship assignment at Avanade asks how AI can be used to improve data quality and to reduce the rejection of data [8]. Keeping rules in step with the standard is one place where these two goals meet.

Language models can also write new candidate rules from a description of the data within seconds, and data-quality tools have started to offer this [9]. Some of these rules are useful. Others apply a wrong business assumption, repeat a control that already exists, or raise alerts on records that are correct. A rule such as "the loan amount must be below five times the income" may look reasonable, but it is not a valid data-quality rule unless the business has such a policy. In addition, the same model can write different code when asked the same question twice [10], and rules that are discovered automatically from data often turn out to be false [11].

For a team that maintains many checks, three questions therefore matter. When the standard changes, which rules must change, and how? Does a new rule catch errors that the existing controls miss, and how many extra alerts does that cost? And would a new or changed rule have behaved well on past data, so that the team can decide whether it may stop data automatically, may only warn, or should only be monitored?

This thesis studies these three questions. It measures how well language models find and update the rules that a new version of a standard affects, compared with the experts' own updates and with simple methods that do not use AI; how much generated rules add beyond independent expert controls, and which method best selects a small set of them; and how new and changed rules behave when they are run on historical data in the order in which that data arrived. The study uses public datasets and publicly documented rules; no Rabobank data is required. A software platform will implement the procedure, and a small data pipeline will show how its results can be used in practice.

2 Pilot work and change of research direction

The direction of this thesis follows from three pilot studies, a short side study and a feasibility check, carried out between September and November 2026 [12, 13, 14]. Section 2.8 lists what was built in each of them. The first two pilots used controlled error injection: values were changed on purpose in a copy of real data, and every change was recorded. Because the location of each error is known, it is possible to count which errors a control finds, which it misses and how many alerts it raises on data that was not changed. The feasibility check and the third pilot instead used real reports and real rule versions without changing them. The figures below are internal pilot results, not independently published findings.

2.1 First pilot: choosing an error detector

The project began with a question about choosing error detectors for tables of data. Error detectors are methods that flag suspicious values without a separate rule being written for every possible error. For example, the CDC method predicts each value from the other columns of the same record and flags values that do not fit the prediction [17]. Earlier benchmark studies had shown large differences between such methods, and had shown that the effort needed to review their alerts is an important part of their cost [18, 19]. Jung et al. showed that the way errors are spread over the data can change how well a detector appears to perform [20].

The pilot therefore separated the type of change made to a value from the mechanism that decides which values are affected. Three mechanisms were tested. Erroneous completely at random (ECAR) means that every value has the same chance of being wrong. Erroneous at random (EAR) means that the chance depends on another recorded field, for example the source system that delivered the record. Erroneous not at random (ENAR) means that the chance depends on the value itself, for example when larger amounts are more likely to contain an error. The pilot applied missing values, invalid types and codes, amounts multiplied by ten and plausible substitutions to three public tables: credit-card clients, bank marketing and wine quality [21, 22, 23]. It compared rule-based and statistical controls with seven detectors, including CDC and Raha [17, 24], and measured how much of their added value remains when a reviewer can check only a limited number of alerts.

The error mechanism changed how many errors a detector found, but not which detector was best. The existing rules already found every missing value, invalid type and invalid code. On harder errors, the strongest additional detector found 11 to 21 of 40 changed values that the rules, the statistical checks and a simple prediction model missed, but it produced 520 to 900 extra alerts per batch of 800 records. With a budget of 20 reviews per 800 records, the extra alerts compete with the alerts of the existing controls for the reviewer's attention. The net gain, the number of extra errors found within those 20 reviews, was on average about 0.4 errors on the bank-marketing data, zero on the credit-card data and negative on the wine data. A negative gain means that the detector's alerts pushed useful alerts of the existing controls out of the review.

2.2 Why the research direction changed

The first pilot was not continued for four reasons. First, the mechanism changed the size of the effect but not the choice of detector, so there was little to select. Second, the question of what an extra detector adds when review time is limited had already been studied [18, 19]. Third, the public tables contain no record of real errors, so the pilot could only show that a detector recovers changes that we made ourselves. Fourth, the tables come without expert rules, so the pilot could not ask what an automatic method adds beyond the rules a real data team already has. The second pilot therefore moved to data that comes with an independent, published set of expert rules.

2.3 Second pilot: rules written by language models

The second pilot used financial reports that US listed companies file with the Securities and Exchange Commission (SEC). The DQC publishes validation rules for these reports together with their code and official test cases [5]. Two small open-weight language models, Qwen2.5-Coder-7B and Qwen3-8B [25, 26], wrote rules in three ways: from the written description of a DQC rule, from an example report, and for types of error that the existing controls missed.

Three results stand out. First, reproducing the behaviour of the official rules was unreliable. For rules published before the models were released, the generated rules produced about 41%

of the messages that the official test cases expect; for rules published afterwards, only 3% and 13% for the two models. Generating the same rule again often gave a very different result. Second, the generated rule sets were too noisy to use as a whole. The existing controls detected 27 of 83 injected errors; the best generated rule set detected six further errors, but raised on average 458 alerts per report that had never been corrected, against 1.8 for the DQC rules. Third, real corrections proved hard to use as ground truth. Of 20 numbers that companies had later corrected, only 9 were clearly wrong in the original report, and 7 of these were accounting restatements rather than data errors.

2.4 Follow-up: judging individual rules

A follow-up experiment looked at the causes and at individual rules instead of complete rule sets. When the model received the official code of nine rules it had failed to reproduce, it fixed three. Most of the remaining failures came from missing domain knowledge, such as information that is only available in the XBRL dictionary of reporting concepts, rather than from missing documentation. When individual generated rules were selected only if they detected errors that the existing controls missed, within a limit of one alert per report, one rule remained. It detected three additional injected errors at about 0.2 alerts per report. A generic filter that ignored the existing controls needed about twice as many alerts for the same gain. This is a small exploratory result and does not establish that the method is better. It does show that useful rules can be hidden inside noisy rule sets, and that selecting them relative to the existing controls is worth testing with stronger models, more data and a proper held-out evaluation.

2.5 Side study and feasibility check

A short side study looked at the EBA's validation rules for the supervisory reports of European banks. Sometimes a rule itself turns out to be wrong, and the EBA then switches it off. The study tested whether a language model could have predicted which rules would later be switched off. The model ranked a faulty rule above a correct one in 64% of the cases, where 50% would be chance, but simply ranking rules by the length of their formula did better (71%), and within a single reporting table the model was close to chance (53%). This direction was therefore not selected [12]. The table of all EBA rules across releases that was built for this study is, however, the basis of the EBA part of the third pilot.

The feasibility check tested whether rules can be evaluated on historical data in the order in which it arrived, as a data pipeline would see it. Twenty-nine SEC reports filed between February 2023 and March 2025 were processed in filing order. The official DQC rules and the generated rules from the second pilot were run on each report, and every alert was linked to its report, its rule, the rule version and the date from which the rule existed. This worked for 98% of the alerts, and the official rules took about two minutes per report. Three results matter for the design. First, 46% of the DQC alerts came from rules that were published only after the report had been filed, so a test with today's rules overstates what a team could have caught at the time. Second, reports from early 2023 need an older release of the DQC rules. Third, none of the 13 checked corrections was flagged by any rule, expert or generated; most of them were accounting restatements, which validation rules are not designed to catch [13].

2.6 Why the research direction changed again

Three observations led to a second change of direction in November 2026. First, generating new data-quality rules with language models has become a crowded research direction (Section 3), and such systems are also offered commercially [9, 50]. Second, the second pilot showed that

the main weakness of generated rules was missing domain knowledge. When a standard changes, much of that knowledge is written down: the publisher states which concepts are deprecated and what replaces them. Third, the feasibility check showed how much the rule set itself changes over time: almost half of today's alerts on reports from 2023 to 2025 came from rules that did not exist when the reports were filed. Real rule sets are maintained continuously, and their published version histories make it possible to measure whether AI can help with this maintenance against the experts' own updates. Rule generation and selection remain part of the thesis (RQ2), but the first question is now whether AI can help keep the existing expert rules up to date.

2.7 Third pilot: updating rules when the standard changes

The third pilot tested whether this direction has enough material, whether the causes of rule changes are documented, and whether the task is within reach of a small local model [14]. The current release of the DQC rules contains a version of every rule for each taxonomy year from 2023 to 2026. Comparing each rule part between consecutive years, after removing comments and formatting, gave 62 changes in the logic of a rule and 56 new rule parts over three yearly updates; another 170 changes only concerned bookkeeping, such as a renamed internal variable. The EBA rule workbook, which covers the releases up to 3.5, gave 1,406 changed formulas in 1,094 rules; 266 of these only changed the notation of row and column codes.

Reading a sample showed where the changes come from. Of 12 DQC changes, 7 to 9 followed from the taxonomy, depending on how changes of scope are counted, and three fixed a mistake in the rule itself. An example of the first kind is the 2026 replacement of the concept `BalanceSheetLocationAxis` by `StatementOfFinancialPositionLocationBalanceAxis`. Of 10 EBA changes, five followed from changed report templates, three were corrections and two only changed the notation. For the DQC, the cause is documented in a form that a program can read: 28 of the 62 logic changes remove a concept that the taxonomy files list as deprecated or removed, and in the 2026 update this was 20 of 28.

A simple change radar, which uses no AI, flagged every rule file that mentions a concept listed as deprecated or removed in the new taxonomy year. In the 2026 update it flagged 15 files, 10 of which the committee had indeed changed, and it found 10 of the 17 changed files. Over all three updates, 55% of the flagged files were changed and 40% of the changed files were flagged. The radar does not yet look at the shared files in which many rules keep their lists of concepts, so it can miss changes that start there. A flagged rule that the committee did not change is also not necessarily a false alarm: the committee sometimes changed a rule only when a deprecated concept was removed from the taxonomy, years after it was deprecated.

Finally, 16 changes that the taxonomy files explain were used to test updates. The model received the old rule and short change notes generated from the taxonomy files, for example "BalanceSheetLocationAxis: deprecated in 2026; replaced by StatementOfFinancialPositionLocationBalanceAxis", and had to return the updated rule. Leaving the rule unchanged was never correct. Replacing each deprecated concept by its named replacement, without AI, produced exactly the official update in 10 of 16 cases. A small local model, Qwen2.5-Coder-7B, given the same input, produced exactly the official update in 14 of 16 cases, in about two minutes per case on an ordinary computer. It solved every case that the replacement solved and four of the six that it did not. In those four, a concept had been removed without a successor, and the model deleted the reference to it, as the committee had done. In one of the two failures, the committee used a new concept that the taxonomy files do not name as the replacement; in the other, the model left the removed concept in the rule. These cases are the easiest kind, because they were selected for being explained by the taxonomy files, and it has not yet been checked whether the model can have seen the newer rule versions during its training.

The pilot shows three things. The material exists: real rule changes with documented causes,

from two independent publishers, with dates. Simple methods already solve part of the task, so the thesis must measure what AI adds beyond them rather than against doing nothing. And the difficult part probably lies elsewhere: in the changes that the change files do not explain, such as corrections of a rule and changed EBA templates, and in finding the affected rules in the first place.

2.8 What was built

Each pilot produced working software, and most of it is reused in the thesis. Table 1 lists what was built, including the parts of pilots that were not continued. All code, results and reports are kept in two project repositories with their full history [15, 16].

2.9 What the pilot work shows

Table 2 summarises how the pilot results shape this proposal. Together they suggest that there is something to gain, both in maintaining existing rules and in adding a few new ones. They also show that the gain only becomes visible when rules are judged one by one, relative to the controls and simple methods that already exist, and when they are tested in a way that matches how they would be used. Whether AI also helps with the rule changes that simple methods cannot handle, and whether new rules find real errors that the existing controls miss, is still open; that is the core of this thesis.

3 Related research and proposed contribution

This section describes six lines of earlier work. For each line it explains what the studies do, how they judge their results, and what they leave open for this thesis. Table 3 at the end of the section compares the most relevant studies.

3.1 Detecting errors and the cost of reviewing alerts

Abedjan et al. ran several error detection tools on real datasets in which the errors were known. Each tool found only part of the errors, and combining tools found more errors but also produced more alerts that a person had to verify [18]. REIN later compared a large number of detection and cleaning methods on the same datasets [19]. The study by Abedjan et al. in particular shows that a new check should be judged by what it adds to existing checks and by the review effort it creates, not by its detections alone. Both studies evaluate data-driven detectors, however, not rules written by language models.

3.2 Language models that write data-quality rules

Alexander et al. asked language models to write data validation tests, small programs that check a dataset, and compared them with a test suite written by one data scientist. They repeated each setup five times to measure how consistent the generated tests were [27]. Akella et al. combine statistical screening of the data with retrieved domain information: the model proposes rules, translates them into executable code, and presents each rule on a readable card for review [28].

LeDQeR asks a model to generate candidate rules for Great Expectations, an open-source tool in which checks are written as “expectations” about a table, starting from an example of an erroneous record. Four filters then remove rules that do not run, that are wrong on labelled data,

Pilot	What was built	What is reused
First pilot (September; not continued)	Error injection for tables with three error mechanisms and four error types; rule-based and statistical controls; adapters for seven error detectors; an evaluation under a review budget; a command-line prototype that registers a source, injects errors into a copy and recommends a detector. In total 990 scenarios, 43 automated tests and an independent recalculation of the reported numbers.	Measuring added value against the existing controls under an alert budget; the injection design.
Second pilot (October)	Download and parsing of SEC reports and the SEC Financial Statement Data Sets; detection of later corrections by comparing original and amended reports; error injection at the level of single XBRL facts; a runner for the official DQC rules with Arelle; local language models on an ordinary laptop; rule generation with execution in a restricted environment and repair; scoring against the official test cases; a rule register with versions and origin.	Almost all of it: register, injection, DQC runner, local models and scoring.
Follow-up (October)	Rule-by-rule qualification of generated rules; a test with the official rule code as input; a check that development and test companies do not overlap.	Selection relative to the existing controls (RQ2).
Side study (October; not continued)	A table of all EBA validation rules across releases, with their deactivation history; simple baselines and a language-model auditor.	The EBA rule table (RQ1).
Feasibility check (November)	A replay engine that runs rules on reports in filing order and links every alert to its report, rule, rule version and availability date.	The basis of the historical test (RQ3).
Third pilot (November)	A corpus of rule changes (DQC per taxonomy year, EBA per release) with a classification of each change; a reader for the FASB change files; the simple change radar; an update test with a local model and two baselines.	The first version of the benchmark of rule changes (RQ1).

Table 1: Software built in the pilot work and its use in the thesis.

that only fit a single record, or that repeat another rule. The rules are scored on benchmark tables in which every error is labelled [29]. AuDQ uses several cooperating "agents", language-model programs that plan steps, call tools and check their own output, to discover rules. The authors also describe how rules can be versioned and traced to their origin, and they let people rate the rules. They note that their datasets have no reference rule set to compare against [30].

Two master's theses address the same task. Schneider guides the model with templates that fix the shape of a rule and compares the output with five to ten expert "golden rules" per dataset, judging by hand whether each golden rule is covered [31]. Siyam compares several commercial models and prompting strategies for generating data-quality rules [32].

Pilot result	Consequence for this thesis
Most DQC logic changes in 2026 followed documented taxonomy changes, and a simple radar found 10 of 17 changed files.	Finding and updating affected rules is the first research question; the simple radar is the baseline to beat (RQ1).
Replacing deprecated concepts by their successors solved 10 of 16 explained updates; a small local model solved 14.	AI is judged on what it adds beyond simple methods, with most weight on the changes that the change files do not explain (RQ1).
Generated rules reproduced older official rules much better than newer ones.	Rule versions are split by publication date relative to the model's release date (RQ1).
Most failures came from missing domain knowledge, not missing documentation.	One condition adds the relevant part of the new standard to the model's input (RQ1).
Whole generated rule sets were too noisy, but one selected rule added detections at 0.2 alerts per report.	Selecting rules relative to the existing controls is the main method for new rules (RQ2).
Most real corrections were accounting restatements, and none of the 13 checked corrections was flagged.	A larger, carefully reviewed set of real errors is built, weighted towards likely data errors (RQ3).
Almost half of today's alerts came from rules that did not yet exist at filing time.	The historical test uses the rule versions that were available at the time (RQ3).

Table 2: Pilot results and their consequences for the proposed research.

These studies establish that rule generation, filtering and human review with language models are existing research directions. They judge the generated rules against labelled benchmark errors, human ratings or a small set of reference rules. None of them compares generated rules with a large, independently maintained rule set using that rule set's own executable tests, none asks what a generated rule adds beyond the controls that are already running, and none considers how rules must change when the data standard changes.

3.3 Selecting a small set of useful checks

Generating many candidate checks is easy; choosing the few that are worth keeping is harder. SPADE addresses this for applications built on language models. It derives many automatic checks on the model's output from the changes developers made to their prompts, and then selects a small subset that still catches the failures developers have labelled, while keeping the share of false failures below a chosen limit [33]. Auto-Validate learns patterns for text columns, such as the format of a date or a product code, from a large collection of tables, and chooses patterns that keep the expected number of false alarms low [34]. LeDQeR's last filter removes a generated rule when another generated rule already covers it [29]. These methods judge candidates on their own or against each other. None of them selects relative to an existing set of expert controls, so they may keep checks that only repeat what those controls already catch.

3.4 Updating code and constraints when their environment changes

Software must be updated when the libraries it depends on change, and this problem has its own benchmarks. CodeUpdateArena tests whether language models can solve programming tasks that require an updated version of a library function. The updates are synthetic, so that no model can have seen them, and the authors report that adding the documentation of the update to the prompt did not enable the open models they tested to use the change [35]. GitChameleon 2.0 tests whether generated code works with a specified version of a library, using executable tests, and reports success rates of about half for strong commercial models [36]. SWE-bench turned

real changes in software projects into a benchmark: each task is a real issue, together with the change that resolved it and the tests that check it [37]. In data management, Volkovs et al. studied continuous data cleaning, in which both the data and the integrity constraints change over time and a system proposes repairs to either [38].

These studies show that keeping code in step with a changing environment is hard for language models and that real version histories make good benchmarks. None of them uses the version history of a real set of data-quality rules, in which both the cause of each change (the new version of the standard) and the experts' answer (the updated rule) are published.

3.5 Checking recurring data and testing on history

Many data sources deliver new data in recurring batches. Redyuk et al. learn from statistics of earlier batches what a normal batch looks like and flag new batches that deviate [39]. Auto-Validate-by-History, developed at Microsoft, uses the history of earlier runs of a recurring pipeline to choose quality constraints with a target false-alarm rate, and was evaluated on about 2,000 production pipelines [40]. Both use history to build and test statistical checks; neither tests rules written by language models against expert rules.

Modern data platforms, often called lakehouses, store data as files and add database features such as versioned tables [41, 42]. In such platforms, new data can first be written to a separate area, checked, and only then published. Tagliabue and Greco argue that AI agents should change a data platform only through such isolated and checked steps [43]. Benchmarks show why care is needed: in ELT-Bench, the best AI agent at the time of publication built only 11.3% of the required data models correctly [44]. In banking, testing a model on past data before trusting it, known as backtesting, has long been standard practice for risk models [45]. We did not find work that backtests generated or updated data-quality rules in this way, or that uses such a test to decide whether a rule may stop data.

3.6 Evaluating code written by language models

Three effects complicate the evaluation of generated rules. Language models perform worse on specialised programming languages with little public code, such as the rule language used for financial reports [46]. Their output can differ between repeated runs [10]. And public rules may have been part of their training data, so a test on such a rule may measure memory rather than skill; benchmarks for generated code therefore separate test items by publication date [47]. For the OLMo models, whose training data is public, it is also possible to search the training data directly for a given text [48, 49].

3.7 Proposed contribution

Table 3 summarises the comparison. Systems that generate data-quality rules with language models also exist commercially and have been patented [9, 50]; the contribution of this thesis is therefore not the idea of such a system. The anticipated contribution is an empirical evaluation with four parts: (1) a public benchmark of real changes to expert data-quality rules, built from the version histories of the DQC and EBA rules, with the documented cause of each change, on which AI and simple methods for finding and updating affected rules are compared, controlling for rules the model may have seen; (2) a method that selects generated rules by what they add beyond the existing expert controls, compared with alternative selection methods at fixed alert budgets; (3) a test of new and updated rules on historical data in the order in which it arrived, in which every rule starts in shadow mode and is promoted step by step, using reviewed real corrections; and (4) a reviewed set of real errors in financial reports. In short, the thesis treats

data-quality rules the way banks treat risk models: before a new or changed rule is trusted, it is tested against an independent reference and on historical data. This claim is provisional and will be reassessed against literature published during the thesis.

Study	What it does	How results are judged	Accounts for existing expert controls	Updates rules when the standard changes	Tested on history
Alexander et al. [27]	Generates data tests	One suite by one data scientist; five repeats	No	No	No
Akella et al. [28]	Generates rules and code with retrieved knowledge	Benchmark data; rule cards	No	No	No
LeDQeR [29]	Generates and filters rules	Labelled benchmark errors	No	No	No
AuDQ [30]	Agents discover rules; provenance	Human ratings	No	No	No
Schneider [31]	Template-guided generation	5 to 10 expert rules per dataset, judged by hand	No	No	No
SPADE [33]	Selects checks on LLM output	Labelled failures; false-failure limit	No	No	No
Auto-Validate-by-History [40]	Learns statistical constraints	Past pipeline runs	No	No	Yes
CodeUpdate-Arena [35]	Updates code for library changes	Programming tasks on synthetic updates	Not applicable	Synthetic changes	No
This proposal	Finds and updates affected rules; generates, selects and backtests rules	Experts' own updates and tests; injected errors; reviewed real corrections	Yes	Yes	Yes

Table 3: Comparison with the most relevant earlier work, as far as reported in the cited sources.

4 Research questions

Main research question: *Can open-weight language models help a data team keep its validation rules up to date and extend them, and what evidence is needed before a new or changed rule may block data, only warn, or only be monitored?*

RQ1. When a data standard publishes a new version, how well can language models identify the expert rules that must change and produce the updated rules, compared with the experts' own updates and with simple methods that do not use AI?

RQ2. How much additional error detection do generated rules provide beyond the existing controls, how do five selection methods compare at fixed alert budgets, and which level of action does the evidence from this selection support?

RQ3. When new and updated rules are run on historical reports in the order in which they were filed, how many reviewed real errors do they detect, how many alerts do they raise, and does a policy that starts every rule in shadow mode and promotes it step by step choose levels of action that still hold on later data?

RQ1 tests whether a model can maintain what experts already wrote; it follows from the third pilot. RQ2 tests whether generated rules can add something that the experts did not write; it follows from the follow-up experiment, in which one selected rule added detections at a low alert cost. RQ3 tests whether the results of RQ1 and RQ2 still hold on real data over time; it follows from the feasibility check.

5 Data sources and reference material

5.1 What "independent expert rules" means

Each public dataset in this study comes with validation rules that were written and published by an organisation outside this project. These organisations maintain the rules for real reporting, document them, and in most cases also publish code or test cases that show what each rule should report (Table 4). Because neither the author nor the language models wrote these rules, they can serve as an independent reference: a generated or updated rule can be compared with them, and they form the existing controls against which any additional value is measured. For RQ1, a second property matters: the rules are published in versions, so that each change from one version to the next, and its cause, can be studied.

Organisation	Who they are	What they publish
XBRL US Data Quality Committee (DQC)	Industry committee for US digital financial reporting	About 200 rules for SEC reports, with code and official test cases, in a separate version for each taxonomy year [5]
Financial Accounting Standards Board (FASB)	US accounting standard setter	The US GAAP taxonomy every year, with files that list deprecated concepts and their replacements [4]
European Banking Authority (EBA)	EU banking authority	Validation rules for the supervisory reports of European banks for each reporting release, with a label that states what changed [6], and a description of the report templates [7]
XBRL International	International standards body for XBRL	Data-quality rules for European company reports, with test cases and expected results [54, 55]
FFIEC and CFPB	US financial regulatory bodies	"Edits" for mortgage reporting, with yearly filing instructions and open-source validation software [56, 57]

Table 4: Organisations that wrote the expert rules and standards used in this study.

5.2 Datasets

D1: US company reports, the DQC rules and their versions. The SEC is the US regulator that receives the reports of listed companies. These reports are filed in XBRL, the eXtensible Business Reporting Language. In XBRL every number is a separate fact with a standard concept name, a reporting date or period, a unit and a level of rounding, for example the concept `Assets` with a value of 340,000,000 US dollars on 31 December 2014. A comparison between facts must therefore respect their accounting meaning, not only their values.

Rule DQC_0004, for example, checks that assets equal liabilities plus equity, and its documentation gives the message "Assets with a value of 340,000,000 is not equal to the total of Liabilities and Equity with a value of 350,000,000" [5]. The DQC rules are written in the rule language XULE and run with the open-source XBRL processor Arelle [5, 51]. Of these rules, 153 have official test cases; 38 were first released after 29 April 2025, the release date of Qwen3. The current release keeps a version of every rule for each taxonomy year from 2023 to 2026, and earlier releases cover earlier years [5]. Together with the FASB change files [4], this gives, for each yearly

update, the old rule, the documented change of the standard and the experts' new rule. The SEC Financial Statement Data Sets show which reports were later amended: from 2023 to mid-2025, 1,294 of 68,875 reports were followed by a corrected version [52]. D1 is the main dataset, because it has the most complete reference and the pilot software already runs on it.

D2: The EBA validation rules and their releases. The EBA publishes the validation rules that the supervisory reports of European banks must pass. Its rule workbook lists every rule in every release, with its formula and a "last change" label such as "Change", "Severity" or "Rename/Split table" [6]. Rule formulas refer to cells in report templates by table, row and column codes, so many changes follow from changed templates; the templates of each release are described in the EBA's data point model [7]. The EBA does not publish executable test cases, so an updated rule is compared with the EBA's own new formula after the notation has been normalised, and a sample is checked by hand. D2 tests whether the results for the DQC also hold for a regulator's rule set in a different domain, and it is the dataset closest to the reporting work of a bank.

D3: European company reports and the ESEF rules. ESEF, the European Single Electronic Format, is the format in which issuers listed on EU regulated markets publish their annual financial reports [53]. Consolidated statements prepared under International Financial Reporting Standards (IFRS) are tagged in this format with Inline XBRL. XBRL International publishes data-quality rules for these reports that are derived from the DQC rules [54]. Of the 25 rules in the current set, 22 have official test reports, and an official file lists the expected results of the rules on 1,739 real European reports from 2022 [55]. The DQC repository also contains its own versions of rules for IFRS and ESEF reports [5]. D3 is used in two ways: as a replication of RQ2 and RQ3 under different accounting standards, and as a second kind of maintenance task, in which a rule written for the US taxonomy must be adapted to the IFRS taxonomy, with the published IFRS or ESEF version as the correct answer.

D4: US mortgage applications and HMDA edits (optional). Under the Home Mortgage Disclosure Act (HMDA), US lenders above certain size thresholds report data on the mortgage applications they receive. The regulators publish "edits", validation rules described in plain language, in yearly filing instructions [56]. Quality edit Q604, for example, checks that the reported county exists in the census list for the reporting year. The regulator's own validation software is open source and contains 318 edit codes and 330 test files [57]. The public loan data has been modified for privacy and has already passed the filing controls, so only suitable fields and controlled error injection will be used. D4 tests RQ2 on a source that is closer to the ordinary tables in a bank's data platform, and is used only if time permits.

D5: Reviewed real errors in SEC reports. Companies sometimes file a corrected version of a report after the original has been published. Comparing the two versions shows which numbers changed. A changed number, however, is not always a data error in the original report. Table 5 shows the three main kinds of change; only the first is the kind of error that a validation rule can be expected to catch. In the second pilot, for example, a company's total assets changed from 7,170,767 to 4,670,767 US dollars because a receivable had been recorded incorrectly in its accounts. The original report correctly showed the accounts as they stood at the time, so no rule applied to that report could have flagged the number. D5 is a reviewed set in which each changed number is assigned to one of these kinds (Section 5.4).

5.3 What counts as the correct answer

To decide whether a rule is good, we need to know what the correct answer is. This reference is often called the ground truth. It differs per research question, because each question asks something different (Table 6).

For RQ1, the question is whether a model finds and updates the rules that the experts changed,

Kind of change	Example	Can a validation rule detect it in the original report?
Data error	A typing error, a reversed sign, or a number reported for the wrong period or under the wrong concept	Yes, in principle
Accounting restatement	The company later finds that a receivable was recorded incorrectly and revises its accounts	Usually not: the original report matched the accounts at the time
Relabelling only	The same number is moved to a different concept	Usually not an error

Table 5: Kinds of change between an original report and its corrected version.

so the experts' new version is the reference. When the DQC moved from the 2025 to the 2026 taxonomy, for example, it changed a set of rules and left the others alone. The rules it changed are the correct answer for the radar, and the new version of each changed rule is the correct answer for the update. Where official test cases exist for the new version, an updated rule is also run on them.

For RQ2, the question is whether a generated rule finds errors that the expert rules miss. The expert rules cannot be the reference here, because they miss errors themselves: in the second pilot, all existing controls together detected 27 of 83 injected errors. Instead, errors are injected into a copy of the data. If, for example, the sign of a company's reported revenue is reversed in the copy, we know exactly which number is wrong and can check which rules flag it.

For RQ3, the question is whether the rules also find real errors. Here the reference is the reviewed set of real corrections (D5), complemented by a random sample of alerts that is checked by hand.

Question	Reference	Main limitation
RQ1: does the model find and update the right rules?	The rules the experts changed, their new versions and, where available, their test cases (D1 to D3)	The experts' version is one correct answer among possibly several, and experts sometimes update a rule late
RQ2: what does the generated rule add?	Injected errors with known locations	Exact, but the errors are not real
RQ3: what would the rule have done on real data?	Reviewed corrections (D5) and a hand-checked sample of alerts	Small; a report that was never corrected is not proven correct

Table 6: Reference used for each research question.

Each reference has limitations. For RQ1, a model may propose an update that differs from the experts' version but is equally correct, so updates that are not identical are run on test cases where possible and a sample is judged by hand. A rule that the experts did not change may still have needed a change, as the third pilot showed, so flagged rules that were not changed are also checked in a sample. For RQ3, a report that was never corrected is not necessarily free of errors. An alert on such a report may point to a real problem that nobody corrected, so it is not automatically a false alarm. Alerts on these reports are therefore treated as a measure of review work, and the share of false alarms is estimated from the hand-checked sample. In addition, as Table 5 shows, a correction is not always an error that could have been seen in the original report, so each reviewed item records whether the problem was visible in the original report.

5.4 Reference material built during the project

Three references do not exist yet and will be built during the project.

Benchmark of rule changes (RQ1). For every change from one version of a standard to the next, the benchmark stores the old rule, the documented change of the standard, the experts' new rule, the test cases where available, the publication date, and the kind of change: caused by the standard, a correction of the rule, or bookkeeping only. The third pilot built a first version for the DQC years 2023 to 2026 and the EBA releases up to 3.5. In the thesis it is extended to earlier DQC years and later EBA releases, and the kind of change is checked by hand for every item; a second person checks a sample. The benchmark will be published, so that other methods can be compared on the same tasks, as SWE-bench is used for changes to software [37].

Reviewed real errors (D5). The target is 300 changed numbers, spread over companies and over kinds of change, with extra weight on changes that are more often data errors, such as a reversed sign, a factor of 1,000 or a shifted reporting period. For each number, a reviewer opens the original and the corrected report, reads the company's explanation of the correction, and records three things: whether the original number was wrong, which kind of change it is (Table 5), and whether the problem was visible in the original report. The pilot used this procedure for 20 numbers, with labels suggested by a language model and checked by the author. In the thesis, the author labels all 300 numbers without such suggestions, following a written guide with examples. A second reviewer, preferably a professional with accounting or XBRL experience, labels the same 100 numbers independently. Agreement is measured with Cohen's kappa, a standard measure that corrects for agreement by chance [58], and disagreements are reported. If no independent reviewer is available, the study will report the lack of an external gold standard and will not present the author's labels as verified fact.

Hand-checked alert sample (RQ3). From the alerts raised in the historical test on reports that were never corrected, a random sample of 200 is checked by hand with the same guide. This shows how many of these alerts point to real problems and how many are false alarms.

6 Experimental design

6.1 RQ1: Finding and updating affected rules

RQ1 consists of two tasks, which are carried out for every version change in the benchmark: from one DQC taxonomy year to the next, from one EBA release to the next, and, in D3, from a US rule to its IFRS counterpart.

Finding the affected rules. The input is the old rule set and the publisher's documentation of the new version: for the DQC, the FASB change files and the new taxonomy; for the EBA, the description of the new templates. The output is a list of rules that must change, each with a reason. Three methods are compared: (1) the simple radar of the third pilot, extended to the shared files in which rules keep lists of concepts; (2) a language model that reads each rule together with the relevant part of the change documentation and decides whether and why it must change; and (3) the model applied only to the rules that the simple radar flags or that use a changed concept indirectly. The measures are precision and recall against the rules the experts changed, and the number of rules a person would have to read to find a given share of the changes.

Updating a rule. For every rule the experts changed, a model writes the updated rule under three input conditions: (A) the old rule and change notes derived from the documentation, as in the third pilot; (B) the same input plus the relevant part of the new standard, such as the definition of a new concept and its place in the taxonomy; and (C) condition B with permission to run the rule on example reports and repair it. Two simple methods serve as baselines: leaving the rule

unchanged, and replacing every deprecated concept by its named successor. The measures are exact agreement with the experts' version after formatting and comments are removed, the result on the official test cases of the new version where they exist, and, for a sample of updates that differ from the experts' version, a judgement by hand of whether the update is also correct. Each task is repeated five times with fixed generation settings, and the results are reported separately for changes caused by the standard and for corrections of the rule itself.

A complication is that the DQC rules are public: their code has been available on GitHub for years. A model that produces the correct update may therefore have seen the new version during its training, in which case the result measures memory rather than skill. Three checks address this. First, version changes are split by publication date: a change published after a model was released cannot have been part of its training data. The second pilot showed why this matters: the generated rules produced about 41% of the expected messages for older rules, against 3% and 13% for newer rules. Second, for the OLMo models of the Allen Institute for AI, whose training data is published, a tool that finds exact matches between a given text and that data makes it possible to check directly whether a rule version was seen [48, 49]. Third, if a new taxonomy year or a new EBA release is published during the thesis, the radar and the update methods are run on it before the experts' updated rules appear, and their output is stored with a date. No model can have seen these answers, which makes this the strictest test. Because the timing of such a release is not under the control of the project, this third check is an extension.

6.2 RQ2: Additional detection and rule selection

RQ2 asks whether generated rules can find errors that the existing controls miss. The experiment has four steps.

First, the existing controls are fixed. They consist of the official expert rules, basic format checks and inexpensive statistical checks learned from other reports, for example a check for an unusual sign or a sudden jump between periods.

Second, errors are injected into a copy of reports that are not known to contain errors. The error types are based on what goes wrong in practice and on earlier work on realistic error models [20]: a reversed sign, a value that is 1,000 times too large because of a wrong unit, a changed digit, a wrong concept or code, swapped reporting periods and a missing value. Where real corrections are available, they determine how large the injected changes are. Running the existing controls on the changed copy shows which error types they catch and which they miss. In the second pilot, for example, the DQC rules caught between 17 and 64% of most error types, but none of the removed values. Each injected change is recorded at the level of the affected fact or field, and results on injected errors are kept separate from conclusions about real error rates.

Third, models generate candidate rules for the error types that the existing controls miss. Candidates are generated on development material and evaluated on companies that were not used during development.

Fourth, the candidates are selected. Five selection methods are compared, using the same candidate pool, the same observations and the same budget: (1) keep every rule that runs; (2) a generic filter based on validity and alert volume, in the style of LeDQeR [29]; (3) a compact selection that balances coverage and false alarms, inspired by SPADE [33]; (4) a language model that rates the plausibility of each rule; and (5) an expert-aware selection that keeps only rules detecting errors that the existing controls miss, while also considering alert volume and repeatability. The fifth method is the main proposed method; its scoring and thresholds will be registered before the final evaluation.

The primary outcome is the number of additional detections at fixed average alert budgets of 0.5, 1 and 2 alerts per report. The budget makes the practical trade-off explicit: a rule that detects

ten extra errors but raises 450 alerts per report is of no use to a team that can review one alert per report. Secondary outcomes include recall, precision where clean labels exist, alerts on reports that were never corrected, runtime, the share of generated rules that fail, and stability across repeats. From the same evidence, each selected rule receives a proposed level of action: block (hold the data), warn (let the data pass with an alert) or monitor (record only). The thresholds for these levels are fixed in advance; whether the levels are reliable is tested in RQ3, and a person always takes the final decision.

This procedure also works outside the experiment. When a team onboards a new data source, it does not know which real errors the data contains, and it does not need to. What it needs to know is which kinds of error its current controls cannot see. Error injection answers that question in the same way that a fire drill tests an alarm without waiting for a real fire. If the existing controls already catch an error type, generated rules have nothing to add for that type, and the study will report this as a result. The pilots show that this differs per error type: in the first pilot, ordinary rules caught every missing value, wrong data type and invalid code, while in the second pilot the existing controls detected 27 of 83 injected errors and none of the removed values.

6.3 RQ3: Testing new and updated rules on historical data

RQ3 asks how new and updated rules would have behaved if they had been in use in the past. The test works like a replay. About 2,000 real SEC reports from 2023 to 2026 are processed week by week, in the order in which they were filed, and the rules are run on each week's reports as if they had been active at that time. Every alert is stored with its report, its rule, the rule version and the date from which the rule existed. Based on the feasibility check, the official rules take about two minutes per report, or roughly 70 hours of computing time in total.

An example shows what this test reveals. Suppose a generated rule states that a company's quarterly revenue cannot be negative. In the replay, it might flag a few reports per year, some of which were later corrected for exactly that number. Such a rule could reasonably be allowed to block data. If it instead flagged dozens of reports per week that were never corrected, it would cost too much review time to block anything, and it should at most warn or be monitored. Only a test on real data over time shows which of these two cases applies.

The replay is run in two forms, because they answer different questions. The historical stress test applies the rules as they exist today to past reports, and shows how a rule behaves on real data from different periods. The time-bound backtest uses only information that was available at the time: the version of the DQC rules that was in force on each filing date, generated rules that were selected using earlier data only, and a model that was released before the replayed period. This shows what a team could really have caught at the time. The difference matters: in the feasibility check, 46% of today's DQC alerts on reports from 2023 to 2025 came from rules that did not yet exist when those reports were filed. All DQC releases are public [5]; if older releases do not run with the current software, the current rule code will be filtered by release date and this will be reported as a limitation.

Three questions are answered with the replay.

Shadow mode and promotion. In practice, a team would not let a new rule block data on its first day. The platform therefore starts every new or updated rule in shadow mode, in which it runs but its alerts are only recorded. A rule is promoted to monitor, then warn, and finally block once it has run on enough data with a low alert rate and, where available, has flagged reviewed errors; it is demoted when its alert rate rises. The replay compares this policy with adopting every selected rule directly at its proposed level. For each approach it measures how many reviewed real errors (D5) would have been flagged, how many alerts per week it raises on reports that were never corrected, what share of these alerts is a false alarm according to the hand-checked sample,

and how often a promoted rule later had to be demoted. The promotion thresholds are fixed in the protocol before the final evaluation.

The cost of not updating rules. For each taxonomy year, the reports of that year are checked once with the rules of the previous year, once with the experts' updated rules and once with the updates proposed by the model. The differences in alerts and in detected errors show what an outdated rule costs in missed checks and in rejected correct data, and how much of this the model's updates recover. This connects RQ1 to the practical goal of reducing the rejection of data. If old rule versions cannot be run on newer reports, only the changed rules are compared.

Stability over time. For every rule, the replay shows whether it keeps the same behaviour from quarter to quarter, and whether a level chosen on earlier data, for example "block" based on 2023 and 2024, is still justified by the results in 2025 and 2026. Recent reports have had less time to be corrected, so these results are reported per period.

As an extension, the platform will also run in a live mode during the last months of the thesis. Each week, newly filed SEC reports are checked with the expert rules and the selected rules, and the alerts are stored with a date before anyone knows which reports will be corrected. A few months later, the corrections filed since then show which alerts pointed to real errors. The number of corrections in such a short period will be small, so this test is reported as a case study rather than as a precise estimate.

6.4 Professional review of the evidence

The platform (Section 8) summarises each new or updated rule on an evidence card: one page that tells a reviewer what the rule does, what changed, and how it performed in the experiments. Table 7 shows what such a card contains. The values in the table are invented for illustration and are not results.

Field	Example content (illustrative only)
Rule in plain words	Quarterly revenue must not be negative, except for the concepts on an exclusion list
Origin	Updated by a local model for the 2026 taxonomy; one concept on the exclusion list was replaced by its successor
Change compared with the previous version	One line changed; shown side by side with the old version
Added beyond existing controls	Catches 14 of 20 injected sign errors in revenue that the expert rules miss
Shadow mode and historical test	0.3 alerts per week on reports that were never corrected; 2 reviewed sign errors flagged before the company corrected them
Stability	Same result in 5 of 5 generations and in every quarter
Proposed level of action	Warn
Decision	Accept, reject or revise, with the reviewer's reason

Table 7: Contents of an evidence card. All values are invented for illustration.

Three to five data-quality or accounting professionals, for example consultants at Avanade, will each review about 30 such cards. For each card they record whether they would accept, reject or revise the rule, at which level of action, and how long the decision took. Their decisions are compared with the levels proposed by the method. This shows whether the evidence is understandable and useful to the people who would approve rules in practice. Reviewer recruitment, ethics approval from the faculty and permission to release derived labels will be arranged before this stage. This part is an extension if reviewers are not available in time.

6.5 Protocol, data splits and statistics

Three measures keep the evaluation fair. First, all choices that could influence the outcome, such as the alert budgets, the selection and promotion thresholds and the outcome measures, are written down in a dated protocol before the final evaluation is run. In the pilots, some limits were chosen after the first results had been seen; the protocol prevents this. Second, methods are developed on one part of the data and evaluated on another: rule selection on one group of companies and evaluation on another group and, where relevant, a later period; rule updates on older version changes and evaluation on newer ones. A method that only works because it was tuned to particular companies or years will then fail the evaluation. Third, every result is reported with a 95% confidence interval, a range that shows how certain the number is given the size of the sample, obtained by resampling over companies or rules. Methods that run on the same data are compared pairwise, and all runs are reported, including failed runs and results that go against the expectations.

7 Models, baselines and reproducibility

The project will use open-weight language models that can be run on controlled infrastructure (Table 8). Such models can be downloaded and run locally, so no data has to be sent to an external service, the exact model version can be stored, and the release date is known, which the contamination analysis requires. The larger experiments will run on the university's computing cluster Hábrók, where software runs in Apptainer containers [59]; the small models remain usable on an ordinary computer, as in the pilots. Self-hosting avoids a mandatory external inference service, although computing and maintenance still have costs.

Model	Released	Role
Qwen2.5-Coder 7B and 32B [25]	2024	Specialised in code; the 7B version provides continuity with the pilots.
Qwen3 8B and 32B [26]	April 2025	General model; the 8B version provides continuity with the pilots.
gpt-oss 20B, optionally 120B [60]	August 2025	Second model family, to test whether results depend on one developer.
OLMo 2 7B and 32B [48]	2024–2025	Public training data, which allows a direct contamination check.

Table 8: Planned models. "B" stands for billions of parameters, a rough measure of model size.

Model versions, generation settings and prompts will be fixed and documented. Every AI method is compared with a method that uses no AI: for finding affected rules, the simple radar; for updating rules, replacing deprecated concepts by their successors; for new rules, the existing expert controls and inexpensive statistical checks. All selection methods see the same generated candidate rules and the same development labels; only the use of the existing expert controls differs. This prevents a stronger generator or extra test knowledge from being mistaken for a better selection method. As a secondary baseline, a language model will also inspect reports directly and point out suspicious facts, without producing reusable rules. This makes the difference between one-time model judgements and reusable, executable checks explicit.

8 Engineering artefact and organisational use

The engineering deliverable is a platform for maintaining and testing data-quality rules before they go live, whether they were written by people or by a language model. It is built from the pilot software (Table 1) and consists of eleven components (Table 9); Figure 1 shows how they work together.

Component	What it does
Rule register	Stores every rule, expert-written or generated, as versioned code with its origin, the version of the standard it was written for, the date from which it was available and its level of action.
Change radar	Reads the documentation of a new version of a standard and lists the rules that are probably affected, with the reason.
Update assistant	Lets a local language model propose an updated rule, shows the difference with the old version, and runs both on example data.
Stress-test engine	Injects realistic errors into a safe copy of the data and shows, per error type, which existing controls catch it: a map of blind spots.
Rule studio	Lets a local language model propose rules for the blind spots, and runs and repairs them in a restricted environment.
Selection	Keeps only rules that add detections beyond the existing controls within the alert budget, and proposes a level of action.
Backtest engine	Replays historical data in time order and reports what each rule, or each version of a rule, would have caught and how many alerts it would have raised.
Shadow mode and promotion	Runs new and changed rules without effect, and proposes promotion or demotion between monitor, warn and block based on the evidence collected.
Quality gate	Runs the approved rules in a data pipeline and publishes each batch, publishes it with warnings, or holds it for review.
Review screen	Shows the radar, the proposed changes, the blind-spot map and the evidence cards, and records accept, reject or revise decisions.
Audit log	Stores every decision with its evidence in an append-only log, so that it can be shown to an auditor.

Table 9: Components of the platform.

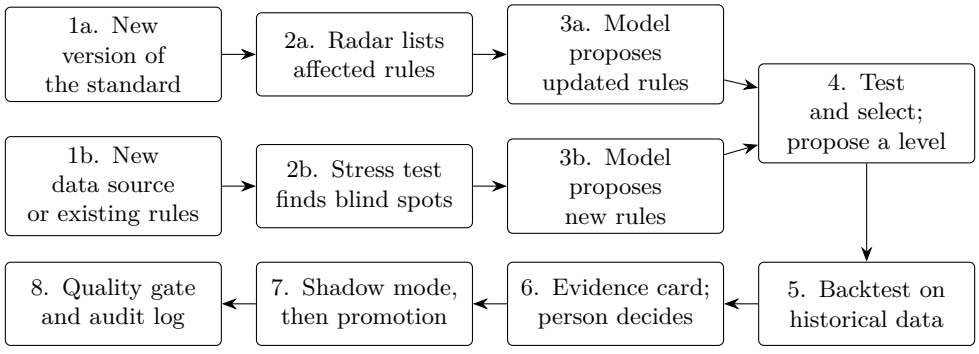


Figure 1: How the platform is used. It never adopts a rule by itself and never changes data.

The pipeline is built with Apache Spark and Delta Lake, the open-source technologies on which platforms such as Databricks are built [61, 42]. New reports arrive in a staging area, the official rules run through Arelle, the approved generated and updated rules run next, and each batch is then published, published with warnings, or held back for review, according to the levels of the rules that fired. Rules in shadow mode run on the same batches, but their alerts are only stored. The audit log is kept separately from the data tables, because older table versions are removed

after a retention period, by default seven days [62]. Generated code runs without production credentials, and the platform never adopts a rule or changes data by itself.

A typical use would look as follows; the numbers in this example are illustrative. A supervisor publishes a new version of a reporting standard in which several items are renamed and one is removed. The radar lists the 15 rules that use these items, with the reason for each. The model proposes an update for each of them, and the tests on example reports show that 12 behave as expected; a person corrects the other three. In the same month, a stress test on the team's data shows that reversed signs on amounts that may legitimately be negative, and single mistyped digits, slip through the existing checks. The model proposes a handful of rules for these gaps, and the selection keeps two. All new and changed rules start in shadow mode. The backtest and the first weeks in shadow mode show, for each rule, how many alerts it raises and which real errors it would have caught. The team promotes the updated rules and one new rule to "warn", and rejects the other new rule. The audit log records why each decision was taken.

This makes the platform useful beyond the thesis. Banks are used to testing risk models on historical data before relying on them [45]; the platform applies the same discipline to data-quality rules, for hand-written rules as well as generated or updated ones. For Avanade and Rabobank, it supports two recurring tasks: updating rule sets when a reporting standard changes, and onboarding new data sources. In both, a team reviews a small number of tested proposals instead of searching and writing by hand, and every new or changed rule is tested on historical data and in shadow mode before it can stop data. Because it runs with a local open-weight model, it can be tried on internal data without sending that data to an external service. It would still require internal security approval, data-specific adapters and integration work before production use, and the project does not promise measurable operational savings before user testing.

The scientific results of the thesis come from RQ1 to RQ3; the platform is the instrument that produces them and the way in which they become usable. To keep the work manageable, the review screen will be simple, and the pipeline will be demonstrated on SEC data only.

9 Scope, feasibility and expected outputs

The project runs for nine months (Table 10). At the end of month 3, progress will be reviewed. The minimum scientific scope is RQ1 on the DQC and EBA rules, RQ2 and RQ3 on the SEC data, at least two model families, a controlled comparison of the selection methods, and an independently reviewed subset of real errors. The ESEF data (D3) is the preferred extension, HMDA (D4) is optional, and the live mode and the test on a newly published standard are extensions. If the programme becomes too large, HMDA, then the live mode, then the depth of the ESEF analysis, and then the number of models will be reduced before the held-out evaluation is compromised.

The expected outputs are a thesis that reports both successful and unsuccessful results, a public benchmark of real rule changes, a reproducible selection and promotion procedure, a reviewed set of real errors subject to permissions, and a tested platform and pipeline with documentation. An outcome in which AI does not help with rule maintenance, or in which generated rules do not provide reliable additional detections, remains informative, provided that the baselines, test data and limitations are reported transparently.

References

[1] L. Tran. Correspondence with the author, 4 September 2026. Unpublished; observations specific to his working context.

Months	Work
1	Protocol, literature update, ethics request, cluster access, review guide; benchmark of rule changes for DQC and EBA; rule register
2–3	RQ1 on DQC and EBA (radar and updates); start of the D5 reviews; progress review at the end of month 3
4	RQ2 on SEC data; ESEF adaptation and replication; data pipeline with quality gate and shadow mode
5–6	RQ3 historical tests, including promotion and outdated rules; completion of D5 and the reviewed alert sample; evidence cards and professional review; start of the live mode
7	Analysis across datasets; robustness checks; publication of the benchmark; demonstration to Avanade and Rabobank
8–9	Writing, documentation and contingency

Table 10: Nine-month plan.

Risk	Response
Simple methods already solve most rule updates	Report where the boundary lies; AI is evaluated on the remaining updates, which tells a team where AI is worth using.
The experts' update is not the only correct one	Run updates on test cases where possible and judge a sample of differing updates by hand.
Generated rules add little, even with larger models	The thesis is a measurement; a clear negative result with its causes is informative.
Few reviewed real errors were visible in the original report	Weight D5 towards correction types that are more often data errors; report RQ3 with uncertainty intervals.
Reports that were never corrected are treated as correct	Use their alerts as a measure of review burden only; estimate false alarms from the reviewed sample.
The historical test uses information from the future	Store availability dates for every rule, rule version and model; run the time-bound form only after the model's release.
No new version of a standard appears during the thesis	The prospective test is an extension; the historical version changes remain the main evaluation.
Building the pipeline takes time from the research	Keep the pipeline small and limited to SEC data; the scientific results come from RQ1 to RQ3.
Reviewers are not available in time	Request ethics approval in month one; report the limitation if no independent review is possible.

Table 11: Main risks and responses.

[2] Basel Committee on Banking Supervision. Principles for effective risk data aggregation and risk reporting (BCBS 239). January 2013. https://www.bis.org/publ/bcbs239.htm.

[3] European Central Bank. Guide on effective risk data aggregation and risk reporting. May 2024. https://doi.org/10.2866/081671.

[4] Financial Accounting Standards Board. US GAAP Financial Reporting Taxonomy, releases 2023 to 2026, including the deprecated-concepts files. https://xbrl.fasb.org/us-gaap/.

[5] XBRL US Data Quality Committee. DQC US rules, documentation and reference tests, version 31. Official repository, accessed November 2026. https://github.com/DataQualityCommittee/dqc_us_rules.

[6] European Banking Authority. Validation rules, rule workbooks for the reporting framework releases up to 3.5, including change labels and deactivation history. https://www.eba.europa.eu/risk-and-data-analysis/reporting/reporting-frameworks/validation-rules-packages.

[7] European Banking Authority. DPM data dictionary. https://www.eba.europa.eu/risk-and-data-analysis/reporting/dpm-data-dictionary.

[8] Avanade. Graduation internship Data & AI, Big Data / Data platform: assignment description, 2026. Unpublished.

[9] T. Rehberger, T. Hütter, L. Ehrlinger, and W. Wöß. Evaluating data quality tools: measurement capabilities and LLM integration. arXiv:2604.09163, 2026.

[10] S. Ouyang, J. M. Zhang, M. Harman, and M. Wang. An empirical study of the non-determinism of ChatGPT in code generation. ACM Transactions on Software Engineering and Methodology, 34(2), 2025. https://doi.org/10.1145/3697010.

[11] A. Martin, E. C. de Almeida, O. Romero, and A. Queralt. How and why false denial constraints are discovered. Proceedings of the VLDB Endowment, 18(10):3477–3489, 2025. https://doi.org/10.14778/3748191.3748209.

[12] T. Oosterbaan. Data-quality controls with and without AI: what two pilot studies showed. Project working report, masterthesis_direction3_pilot, October 2026. Unpublished; numerical results reported by the author.

[13] T. Oosterbaan. Backtest feasibility check. Project working report, masterthesis_direction3_pilot, docs/backtest_feasibility.md, November 2026. Unpublished.

[14] T. Oosterbaan. Pilot B: rule maintenance when a data standard changes. Project working report, masterthesis_direction3_pilot, docs/pilot_b_rule_maintenance.md, November 2026. Unpublished.

[15] T. Oosterbaan. First pilot: code, configurations and results. Repository masterthesis_pilot_final, 2026. Private.

[16] T. Oosterbaan. Second and third pilot, side study and feasibility check: code and results. Repository masterthesis_direction3_pilot, 2026. Private.

[17] S. Jäger and F. Biessmann. From data imputation to data cleaning: automated cleaning of tabular data improves downstream predictive performance. AISTATS, PMLR 238:3394–3402, 2024.

[18] Z. Abedjan et al. Detecting data errors: where are we and what needs to be done? Proceedings of the VLDB Endowment, 9(12):993–1004, 2016. https://doi.org/10.14778/2994509.2994518.

[19] M. Abdelaal, C. Hammacher, and H. Schöning. REIN: a comprehensive benchmark framework for data cleaning methods in ML pipelines. EDBT, 2023. https://doi.org/10.48786/edbt.2023.43.

[20] P. Jung, S. Jäger, N. Chandler, and F. Biessmann. Towards realistic error models for tabular data. ACM Journal of Data and Information Quality, 17(4), 2025. https://doi.org/10.1145/3774914.

[21] I.-C. Yeh. Default of credit card clients. UCI Machine Learning Repository, 2009. https://doi.org/10.24432/C55S3H.

[22] S. Moro, P. Rita, and P. Cortez. Bank marketing. UCI Machine Learning Repository, 2014. https://doi.org/10.24432/C5K306.

[23] P. Cortez, A. Cerdeira, F. Almeida, T. Matos, and J. Reis. Wine quality. UCI Machine Learning Repository, 2009. https://doi.org/10.24432/C56S3T.

[24] M. Mahdavi et al. Raha: a configuration-free error detection system. SIGMOD, 865–882, 2019. https://doi.org/10.1145/3299869.3324956.

[25] B. Hui et al. Qwen2.5-Coder technical report. arXiv:2409.12186, 2024.

[26] A. Yang et al. Qwen3 technical report. arXiv:2505.09388, 2025.

[27] R. Alexander et al. Evaluating the decency and consistency of data validation tests generated by LLMs. arXiv:2310.01402, 2023.

[28] A. Akella, A. Kaul, K. Narayanam, and S. Mehta. Quality assessment of tabular data using large language models and code generation. EMNLP Industry Track, 2025. https://doi.org/10.18653/v1/2025.emnlp-industry.183.

[29] A.-C. Glock et al. Data quality rule generation with LLMs. arXiv:2609.06053, September 2026.

[30] J. Biggiogera et al. AuDQ: auditable data quality through agentic rule discovery. EMNLP Industry Track, 2026. https://aclanthology.org/2026.emnlp-industry.113/.

[31] A. Schneider. Template-guided rule generation and evaluation for data quality using large language models. Diploma thesis, TU Wien, 2025. https://doi.org/10.34726/hss.2025.135903.

[32] S. Siyam. Evaluating the effectiveness of LLMs and prompting techniques in generating data quality rules. Master's thesis, University of Jyväskylä, 2025. https://jyx.jyu.fi/handle/123456789/102965.

[33] S. Shankar et al. SPADE: synthesizing data quality assertions for large language model pipelines. Proceedings of the VLDB Endowment, 17(12), 2024. https://doi.org/10.14778/3685800.3685835.

[34] J. Song and Y. He. Auto-Validate: unsupervised data validation using data-domain patterns inferred

from data lakes. SIGMOD, 2021. https://doi.org/10.1145/3448016.3457250.
[35] Z. L. Liu, S. Pandit, X. Ye, E. Choi, and G. Durrett. CodeUpdateArena: benchmarking knowledge editing on API updates. arXiv:2407.06249, 2024.
[36] D. Misra, N. Islah, V. May, B. Rauby, Z. Wang, J. Gehring, A. Orvieto, M. Chaudhary, E. B. Muller, I. Rish, S. E. Kahou, and M. Caccia. GitChameleon 2.0: evaluating AI code generation against Python library version incompatibilities. arXiv:2507.12367, 2025.
[37] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan. SWE-bench: can language models resolve real-world GitHub issues? ICLR, 2024. arXiv:2310.06770.
[38] M. Volkovs, F. Chiang, J. Szlichta, and R. J. Miller. Continuous data cleaning. ICDE, 244–255, 2014. https://doi.org/10.1109/ICDE.2014.6816655.
[39] S. Redyuk, Z. Kaoudi, V. Markl, and S. Schelter. Automating data quality validation for dynamic data ingestion. EDBT, 2021.
[40] D. Tu et al. Auto-Validate by-History: auto-program data quality constraints to validate recurring data pipelines. KDD, 2023. https://doi.org/10.1145/3580305.3599776.
[41] M. Armbrust, A. Ghodsi, R. Xin, and M. Zaharia. Lakehouse: a new generation of open platforms that unify data warehousing and advanced analytics. CIDR, 2021.
[42] M. Armbrust et al. Delta Lake: high-performance ACID table storage over cloud object stores. Proceedings of the VLDB Endowment, 13(12):3411–3424, 2020. https://doi.org/10.14778/3415478.3415560.
[43] J. Tagliabue and C. Greco. Safe, untrusted, "proof-carrying" AI agents: toward the agentic lakehouse. IEEE Big Data Workshop on Secure and Safe AI Agents for Big Data Infrastructures, 2025. arXiv:2510.09567.
[44] T. Jin, Y. Zhu, and D. Kang. ELT-Bench: an end-to-end benchmark for evaluating AI agents on ELT pipelines. Proceedings of the VLDB Endowment, 19, 2025. arXiv:2504.04808.
[45] Basel Committee on Banking Supervision. Supervisory framework for the use of "backtesting" in conjunction with the internal models approach to market risk capital requirements. January 1996. https://www.bis.org/publ/bcbs22.htm.
[46] S. Joel, J. J. Wu, and F. H. Fard. A survey on LLM-based code generation for low-resource and domain-specific programming languages. ACM Transactions on Software Engineering and Methodology (accepted). arXiv:2410.03981.
[47] N. Jain et al. LiveCodeBench: holistic and contamination free evaluation of large language models for code. ICLR, 2025. arXiv:2403.07974.
[48] Team OLMo. 2 OLMo 2 Furious. arXiv:2501.00656, 2024.
[49] J. Liu et al. OLMoTrace: tracing language model outputs back to trillions of training tokens. ACL System Demonstrations, 2025. arXiv:2504.07096.
[50] Systems and methods for automatically ensuring data quality using LLMs. US Patent 12,632,447.
[51] Arelle. Open-source XBRL processor. https://arelle.org.
[52] US Securities and Exchange Commission. Financial Statement Data Sets, 2023q1 to 2025q2. https://www.sec.gov/data-research/sec-markets-data/financial-statement-data-sets.
[53] European Securities and Markets Authority. Enforcement of corporate reporting: ESEF reporting. https://www.esma.europa.eu/issuer-disclosure/enforcement-corporate-reporting.
[54] XBRL International. ESEF Data Quality Rules. https://esef-dqr.xbrl.org.
[55] XBRL International. ESEF DQR conformance suite. https://gitlab.xbrl.org/xii/dqr-conformance.
[56] FFIEC and CFPB. 2026 HMDA Filing Instructions Guide. https://ffiec.cfpb.gov/documentation/fig/2026/overview.
[57] Consumer Financial Protection Bureau. HMDA Platform, open-source filing and validation software. https://github.com/cfpb/hmda-platform.
[58] J. Cohen. A coefficient of agreement for nominal scales. Educational and Psychological Measurement, 20(1):37–46, 1960.
[59] University of Groningen. Hábrók cluster documentation: Apptainer containers. https://wiki.hpc.rug.nl/habrok/examples/apptainer.
[60] OpenAI. gpt-oss-120b and gpt-oss-20b model card. arXiv:2508.10925, 2025.
[61] M. Zaharia et al. Apache Spark: a unified engine for big data processing. Communications of the ACM, 59(11):56–65, 2016. https://doi.org/10.1145/2934664.
[62] Databricks. VACUUM, SQL language reference. https://docs.databricks.com/aws/sql/language-manual/delta-vacuum.